Hitachi Consulting
We make it happen. Better.

Graph Analytics

Basic Theory and Applications

Khalid M. Salama, Ph.D.
Business Insights & Analytics
Hitachi Consulting UK

 | © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

Outline

- Overview on Graphs
- Path Analytics
- Connectivity Analytics
- Community Analytics
- Centrality Analytics
- Pattern Matching
- Parallel Programming Model for Graphs
- Applied Graph Analytics
- Useful Resources

 | © Copyright 2015 Hitachi Consulting

Introduction

Graph Analytics and Databases

Graph Analytics - *"Built on the mathematics of graph theory, graph analytics help to understand, codify, and visualize* ***relationships that exist between objects*** *in a given domain context, in order to* ***uncover insights*** *about the* ***structures and patterns*** *of the objects relationships."*

Graph Databases – *"A NoSQL family of data stores that is* ***optimized to store, model, and process data in a graphical form****, as well as answering graph-related* ***queries efficiently****."*

 | © Copyright 2015 Hitachi Consulting

Graphs Overview

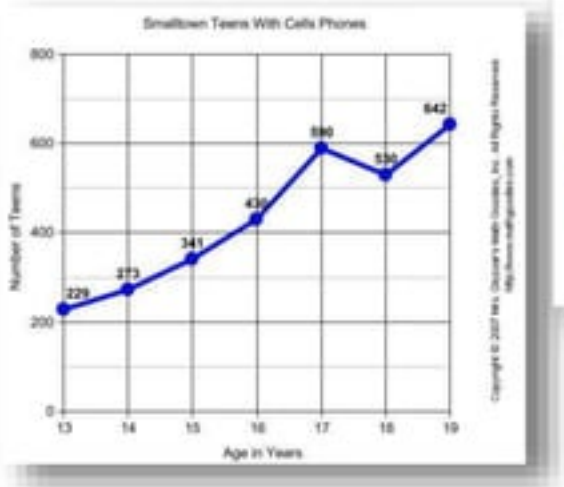
 | © Copyright 2015 Hitachi Consulting

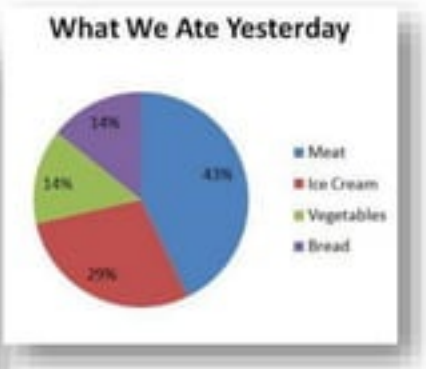
What is NOT a Graph?

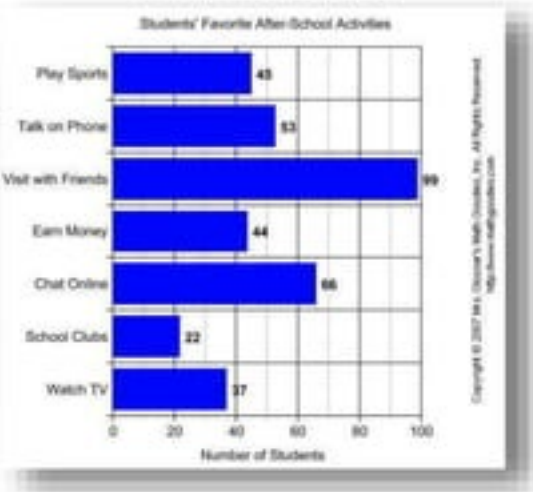
Basic Concepts

Hitachi Consulting
We make it happen. Better.

These are **NOT** graphs!







These are **charts!**

© Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

What is a Graph?

Basic Concepts

*In computing, a graph is abstract **data structure** that represents set objects and their relationships as **vertices and edges**, and supports a number of graph-related **operations***

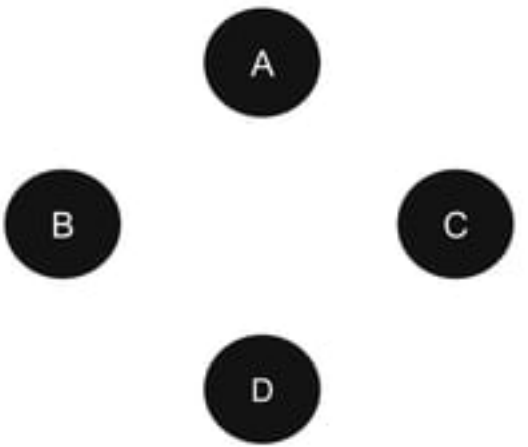
 | © Copyright 2015 Hitachi Consulting

What is a Graph?

Basic Concepts

*In computing, a graph is abstract **data structure** that represents set objects and their relationships as **vertices and edges**, and supports a number of graph-related **operations***

- **Objects (nodes)**: {A, B, C, D}



 | © Copyright 2015 Hitachi Consulting

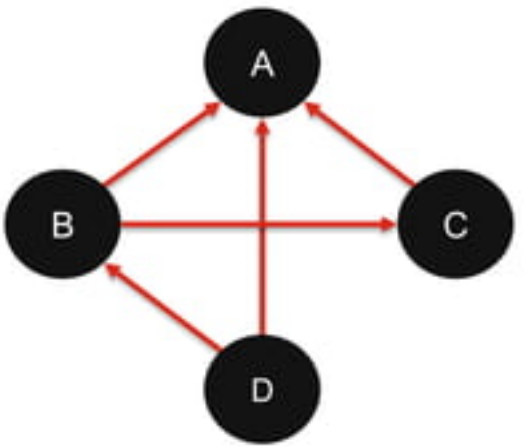
Hitachi Consulting
We make it happen. Better.

What is a Graph?

Basic Concepts

*In computing, a graph is abstract **data structure** that represents set objects and their relationships as **vertices and edges**, and supports a number of graph-related **operations***

- **Objects (nodes)**: {A, B, C, D}
- **Relationships (edges):** {(D,B),(D,A),(B,C),(B,A),(C,A)}



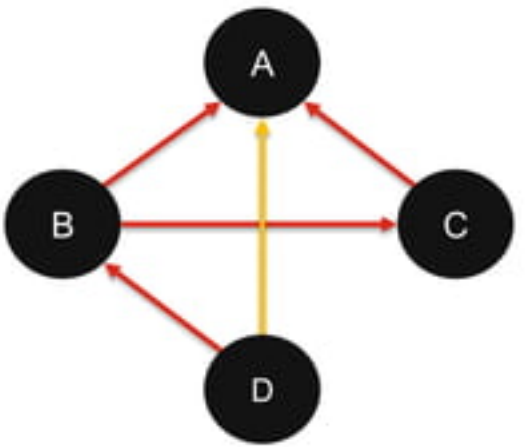
 | © Copyright 2015 Hitachi Consulting

What is a Graph?

Basic Concepts

*In computing, a graph is abstract **data structure** that represents set objects and their relationships as **vertices and edges**, and supports a number of graph-related **operations***

- **Objects (nodes)**: {A, B, C, D}
- **Relationships (edges):** {(D,B),(D,A),(B,C),(B,A),(C,A)}
- **Operation**: shortest path between D and A



 | © Copyright 2015 Hitachi Consulting

What is a Graph?

Graph operation examples

Hitachi Consulting
We make it happen. Better.

- graph.GetNodes(<condition>)
- graph.GetEdges(<condition>)
- graph.AddNode(node)
- graph.AddEdge(node1,node2)
- graph.AddEdge(edge)
- graph.RemoveNode(node)
- graph.GetShortestPath(node1,node2)
- graph.Neighbours(node,level)
- graph.GetDistance(node1,node2)

- node.GetParents()
- node.GetChildren()
- node.GetAncestors(level)
- node.GetDescendants(level)
- node.IsAncestorTo(node2)
- node.IsDescendant(node2)
- node.AddParent(parentNode)
- node.AddChild(childeNode)
- node.IsReachable(node2)

 © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

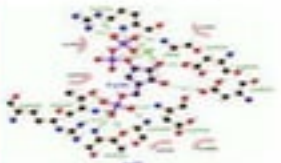
What is a Graph?

Real-world examples…

Social Media – Twitter

- Identify groups (communities) and group interactions
- Find influencers in community
- Extract topic interests

Biology – Biological Entities Networks



- Discover unknown relationships (gene/ protein to disease, disease to disease, cure to disease, etc.)
- Exploratory Data Analysis & anomaly detection

Geo IS – Smart Cities

- Coverage analysis
- Traffic flow, congestion estimation, routing
- Failure Impact analysis

Reasoning – Predictive Maintenance

- Predict the next state given the current (and previous state(s))
- Compute the probability of sequence of event

Hitachi Consulting
We make it happen. Better.

Why Graphs?

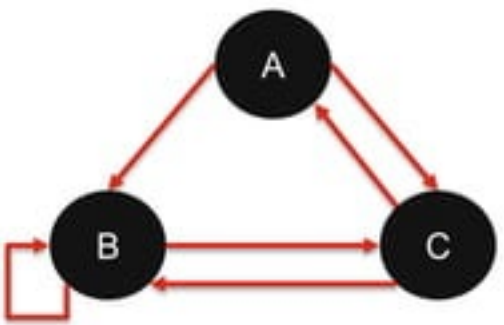
Importance of graph data structures

Hitachi Consulting
We make it happen. Better.

Graph Types

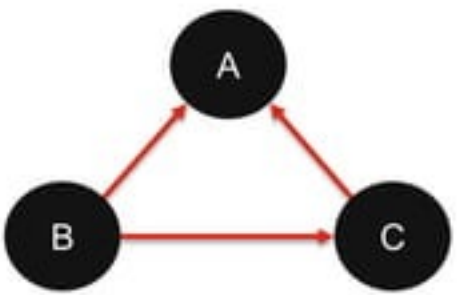
Directionality and circulation

Directed Graphs



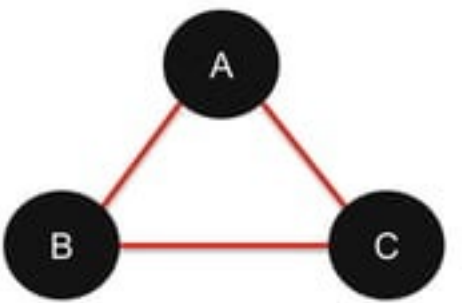
State-transition models

Directed Acyclic Graphs



Dependency networks

Undirected Graphs

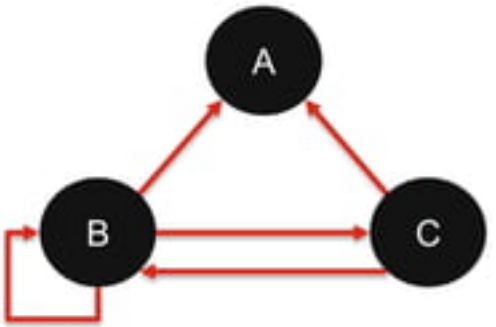


Connectivity networks

 | © Copyright 2015 Hitachi Consulting

Simple Graph Representation

Adjacency Matrix

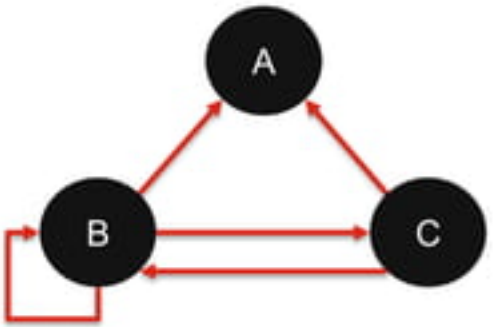


 | © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

Simple Graph Representation

Adjacency Matrix



To

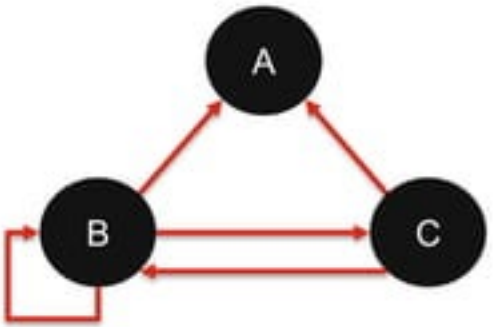
From	A	B	C
A	0	0	0
B	1	1	1
C	1	1	0

 | © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

Simple Graph Representation

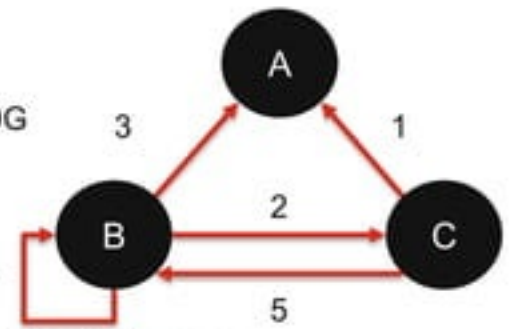
Adjacency Matrix



To

From	A	B	C
A	0	0	0
B	1	1	1
C	1	1	0

Weighted DG



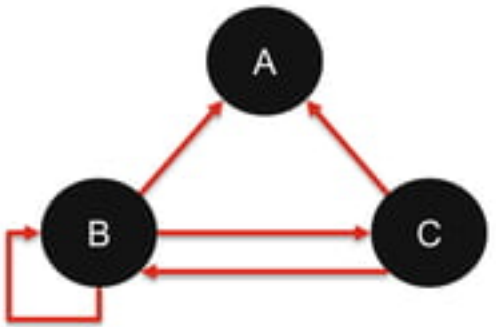
From	A	B	C
A	0	0	0
B	3	4	2
C	1	5	0

 | © Copyright 2015 Hitachi Consulting

Simple Graph Representation

Edge Table

Hitachi Consulting
We make it happen. Better.



FROM	TO	WEIGHT
B	A	..
B	C	..
B	B	..
C	B	..
C	A	..

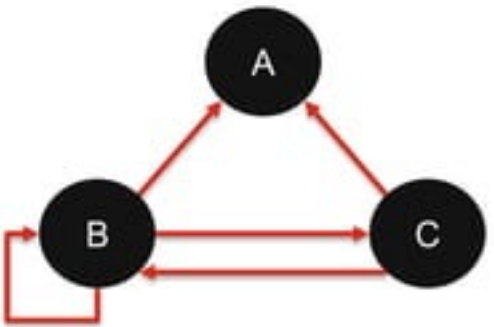
Useful in Relational Databases

 | © Copyright 2015 Hitachi Consulting

Simple Graph Representation

Adjacency list

Hitachi Consulting
We make it happen. Better.



Node	IN	OUT
A	B,C	-
B	B,C	A,C
C	B	A,B

Useful in MapReduce

 | © Copyright 2015 Hitachi Consulting

Label Property Graph Model

Defining information-rich graphs

In a simple model, a graph consist of:

- A set of vertices (nodes)
- A set of edges (each connecting two nodes)

 | © Copyright 2015 Hitachi Consulting

Label Property Graph Model

Defining information-rich graphs

In a simple model, a graph consist of:

- A set of vertices (nodes)
- A set of edges (each connecting two nodes)

In the Label Property Graph Model, each element (vertex/edge) has:

- Unique Identifier
- Class (label)
- A set of Key/Value pairs (properties)

 | © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

Label Property Graph Model

Defining information-rich graphs

In a simple model, a graph consist of:

- A set of vertices (nodes)
- A set of edges (each connecting two nodes)

In the Label Property Graph Model, each element (vertex/edge) has:

- Unique Identifier
- Class (label)
- A set of Key/Value pairs (properties)

 | © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

Label Property Graph Model

Defining information-rich graphs

In a simple model, a graph consist of:

- A set of vertices (nodes)
- A set of edges (each connecting two nodes)

In the Label Property Graph Model, each element (vertex/edge) has:

- Unique Identifier
- Class (label)
- A set of Key/Value pairs (properties)

 | © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

Label Property Graph Model

Defining information-rich graphs

In a simple model, a graph consist of:

- A set of vertices (nodes)
- A set of edges (each connecting two nodes)

In the Label Property Graph Model, each element (vertex/edge) has:

- Unique Identifier
- Class (label)
- A set of Key/Value pairs (properties)

 | © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

Label Property Graph Model

Defining information-rich graphs

In a simple model, a graph consist of:

- A set of vertices (nodes)
- A set of edges (each connecting two nodes)

In the Label Property Graph Model, each element (vertex/edge) has:

- Unique Identifier
- Class (label)
- A set of Key/Value pairs (properties)

 | © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

Label Property Graph Model

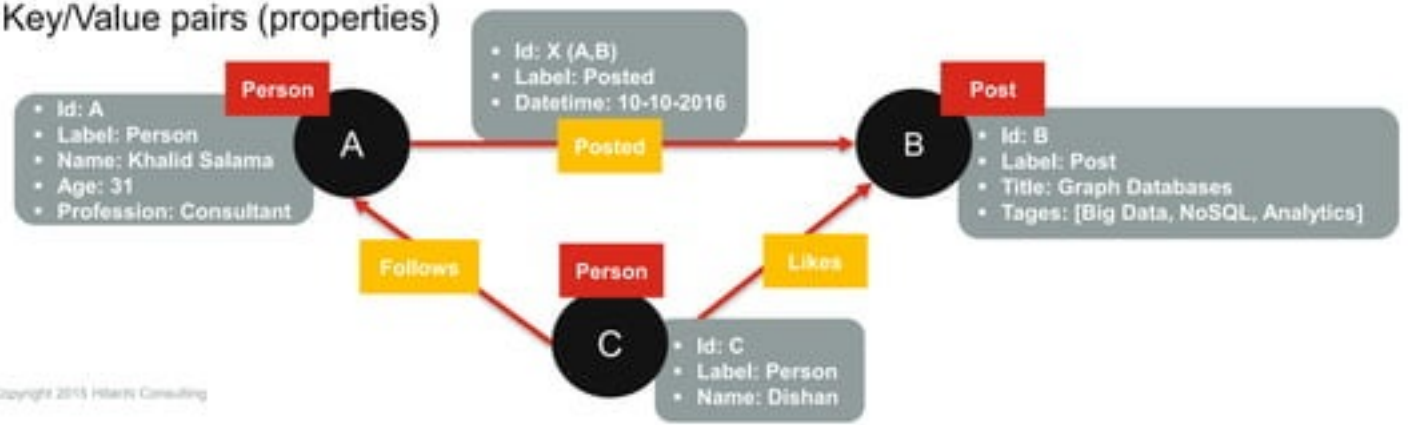
Defining information-rich graphs

In a simple model, a graph consist of:

- A set of vertices (nodes)
- A set of edges (each connecting two nodes)

In the Label Property Graph Model, each element (vertex/edge) has:

- Unique Identifier
- Class (label)
- A set of Key/Value pairs (properties)



 | © Copyright 2015 Hitachi Consulting

Types of Graphs Analytics

 | © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

Types of Graph Analytics

Relationships Analytics

Connectivity Analytical

Path Analytics & Traversing

Community Analytics

Centrality Analytics

Pattern Matching

 | © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

Connectivity Analytics

 | © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

Connectivity Analytics

Graph structural analysis

How big is the graph?

Volume – Number of edges increases quadraticly with respect to number of nodes

Velocity – How frequent a new vertex or edge is added to the graph

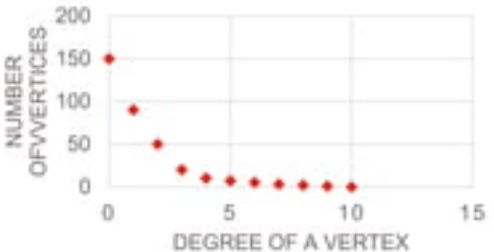
Degree

- In-degree of a vertex: number of edges pointing to the vertex (parents)
- Out-degree of a vertex: number of edges point out of the vertex (children)
- Degree of a vertex: number of neighbour of a vertex in an undirected graph

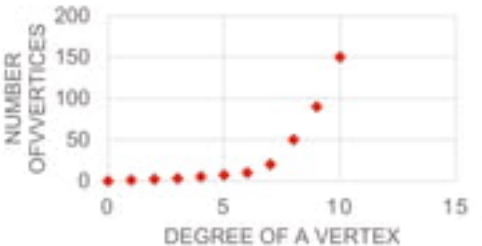
Connectivity Analytics

Graph structural analysis

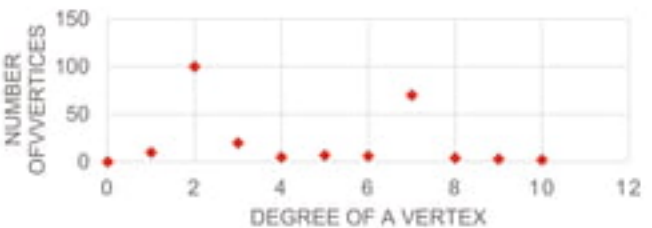
Degree Histogram – describes the skewness of the degree distribution in a graph



Exponentially unlikely to find a vertex with increased degree



In some case, it is more likely to find more vertices with high number of edges



Or it can be multi-modal

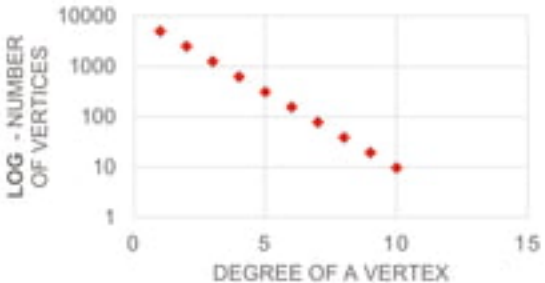
 | © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

Connectivity Analytics

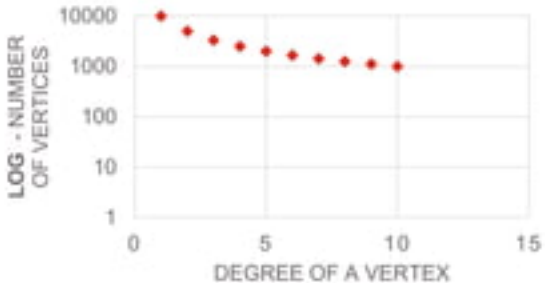
Graph structural analysis

Degree Histogram – Random vs Natural Graphs



In random graphs, exponentially unlikely to find a vertex with increased degree

Exponential Distribution $n(d) \simeq c\left(\frac{1}{2}\right)^d$



In some case, it is more likely to find more vertices with high number of edges

ZipF Distribution $n(d) \simeq \frac{1}{d^r}$

A vertex with higher degree (more connections) is more likely to get a new edge, compared to less connected vertices – Social Networks

 | © Copyright 2015 Hitachi Consulting

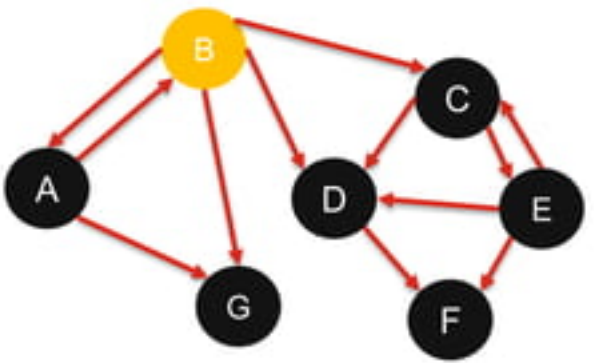
Hitachi Consulting
We make it happen. Better.

Connectivity Analytics

Graph structural analysis

- **Highly connected nodes** – nodes with high In/Out-Degree.
- **Graph Robustness** – how easy to break the graph by removing a few nodes/edges (Built-in Redundancy)
  - **Connectivity Coefficient**: minimum number of nodes you need to remove to disconnect a graph (E.g. node B)
    - Useful in network fragility analysis and social media advertising
  - **Connectivity**: X is reachable from Y **OR** Y is reachable from X
  - **Strong Connectivity**: X is reachable from Y **AND** Y is reachable from X
    - High degree nodes make the network more vulnerable.

- Graph Comparison – how similar graph G1 to G2?
  - Number of nodes
  - Number of edges
  - Ratio of Nodes to Edges
  - In/Out Degree Histogram
  - Connectivity Coefficient



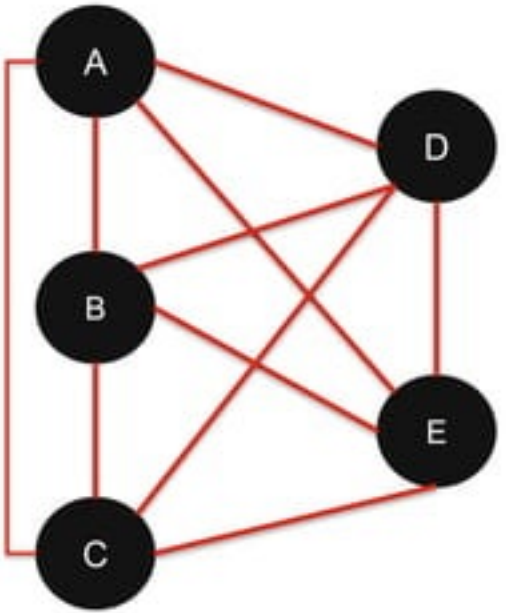
 | © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

Connectivity Analytics

Graph structural analysis

- **Fully connected graph**: Each node has edges to all the other nodes (usually undirected graph)
  - Can we find subgraphs, in a given graphs, that are fully connected? **(Cliques)**



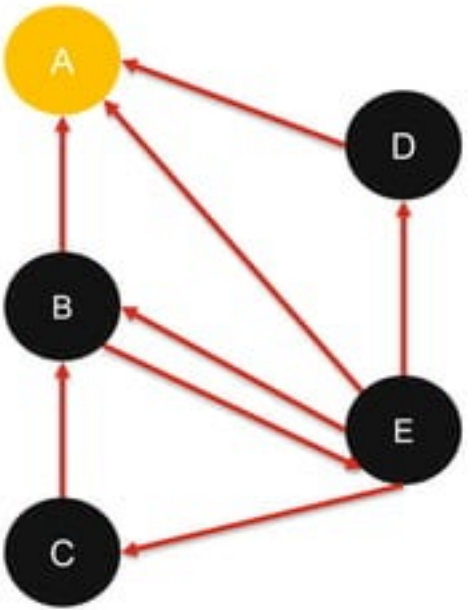
 | © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

Connectivity Analytics

Graph structural analysis

- **Fully connected graph**: Each node has edges to all the other nodes
- **Terminal node**: A node with no outgoing edges



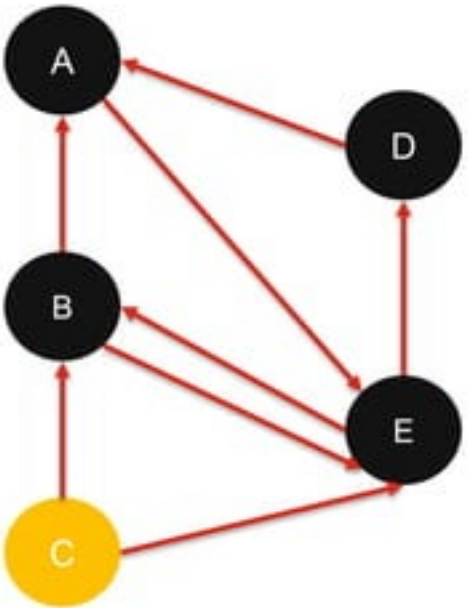
 | © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

Connectivity Analytics

Graph structural analysis

- **Fully connected graph**: Each node has edges to all the other nodes
- **Terminal node**: A node with no outgoing edges
- **Unreachable node**: A node no ingoing edges



 | © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

Connectivity Analytics

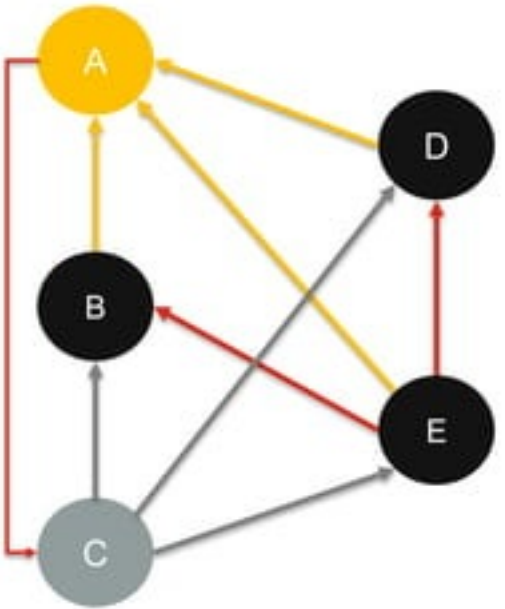
Graph structural analysis

- **Fully connected graph**: Each node has edges to all the other nodes
- **Terminal node**: A node with no outgoing edges
- **Unreachable node**: A node no ingoing edges
- **Hub vs. Authorities:** High In-degree vs High Out-degree

  A is a hub node, C is an authority node

  E.g.: Social Networks: Talkers vs. Listener

  E.g.: Web structure



 | © Copyright 2015 Hitachi Consulting

Path Analytics

 | © Copyright 2015 Hitachi Consulting

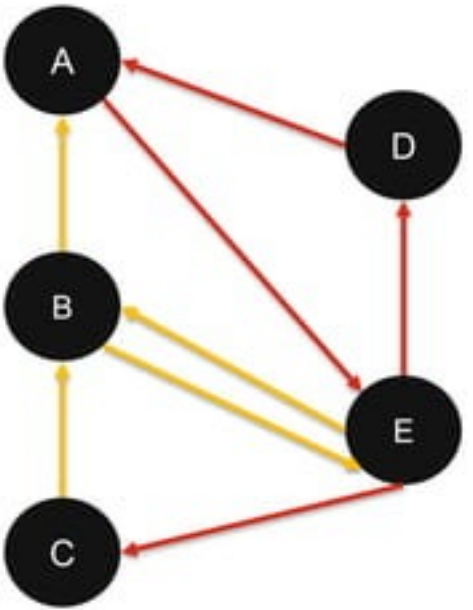
Hitachi Consulting
We make it happen. Better.

Path Analytics & Graph Traversing

Concepts and operations

- **Path**: A set of (ordered) edges between node x and node y

  C → B → E→ B → A



 | © Copyright 2015 Hitachi Consulting

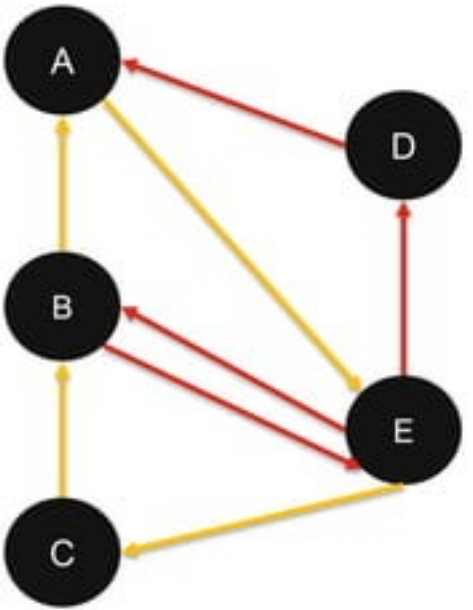
Hitachi Consulting
We make it happen. Better.

Path Analytics & Graph Traversing

Concepts and operations

- **Path**: A set of (ordered) edges between node x and node y
- **Cycle**: A path where the start and the end nodes are the same

  E→C→B→A→E



 | © Copyright 2015 Hitachi Consulting

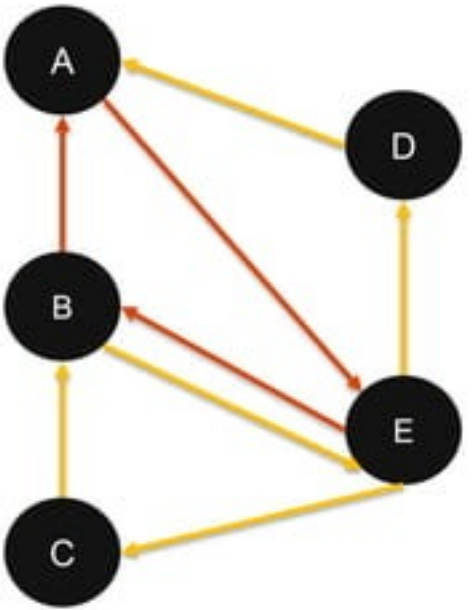
Hitachi Consulting
We make it happen. Better.

Path Analytics & Graph Traversing

Concepts and operations

- **Path**: A set of (ordered) edges between node x and node y
- **Cycle**: A path where the start and the end nodes are the same
- **Trail**: A path with no repeated edges

  E→C→B→E→D→A



 | © Copyright 2015 Hitachi Consulting

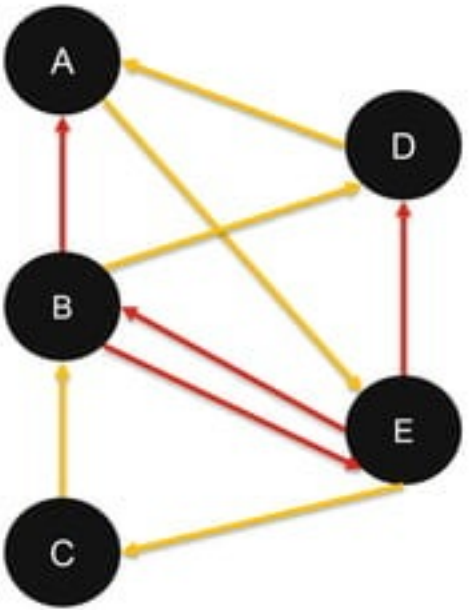
Hitachi Consulting
We make it happen. Better.

Path Analytics & Graph Traversing

Concepts and operations

- **Path**: A set of (ordered) edges between node x and node y
- **Cycle**: A path where the start and the end nodes are the same
- **Trail**: A path with no repeated edges
- **Tour**: A cycle traversing all the nodes, only once.

  D→A→E→C→B→D



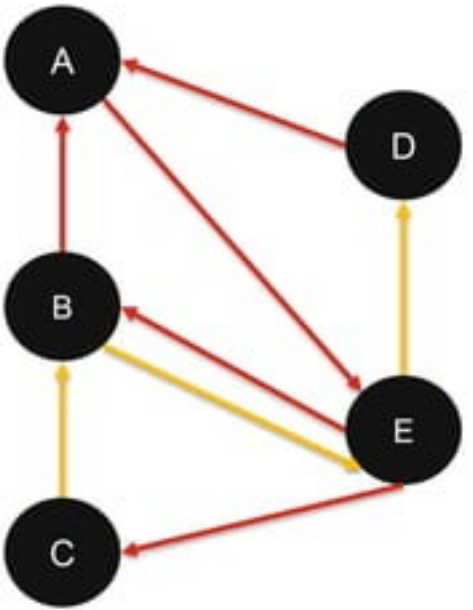
 | © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

Path Analytics & Graph Traversing

Concepts and operations

- **Path**: A set of (ordered) edges between node x and node y
- **Cycle**: A path where the start and the end nodes are the same
- **Trail**: A path with no repeated edges
- **Tour**: A cycle traversing all the nodes, only once.
- **Reachability**: Can we reach node D from node C?



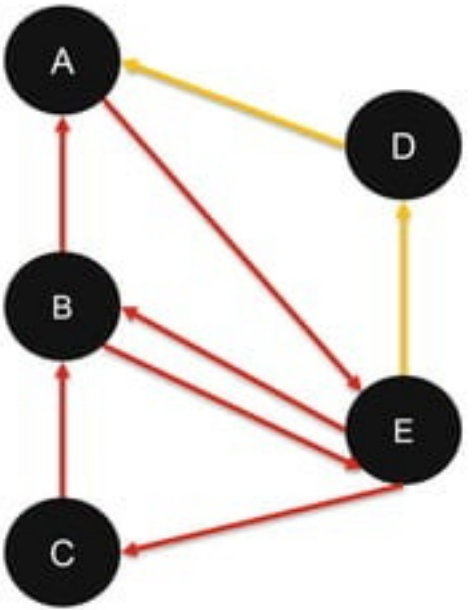
 | © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

Path Analytics & Graph Traversing

Concepts and operations

- **Path**: A set of (ordered) edges between node x and node y
- **Cycle**: A path where the start and the end nodes are the same
- **Trail**: A path with no repeated edges
- **Tour**: A cycle traversing all the nodes, only once.
- **Reachability**: Can we reach node D from node C?
- **Shortest path**: minimum steps (edges) between two nodes
  - Breadth-First Search
  - Dijkstra's algorithm



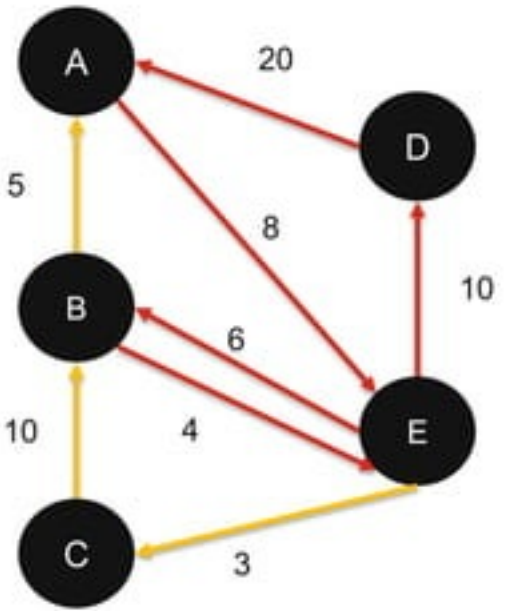
 | © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

Path Analytics & Graph Traversing

Concepts and operations

- **Path**: A set of (ordered) edges between node x and node y
- **Cycle**: A path where the start and the end nodes are the same
- **Trail**: A path with no repeated edges
- **Tour**: A cycle traversing all the nodes, only once.
- **Reachability**: Can we reach node D from node C?
- **Shortest path**: minimum steps (edges) between two nodes
  - Breadth-First Search
  - Dijkstra's algorithm
- **Best path** (weighted graph): path that minimize total weight
  - Optimize a given function
  - Satisfy given constrains



 | © Copyright 2015 Hitachi Consulting

Path Analytics & Graph Traversing

Concepts and operations

- **Path**: A set of (ordered) edges between node x and node y
- **Cycle**: A path where the start and the end nodes are the same
- **Trail**: A path with no repeated edges
- **Tour**: A cycle traversing all the nodes, only once.
- **Reachability**: Can we reach node D from node C?
- **Shortest path**: minimum steps (edges) between two nodes
  - Breadth-First Search
  - Dijkstra's algorithm
- **Best path** (weighted graph): path that minimize total weight
  - Optimize a given function
  - Satisfy given constrains
- **Graph Diameter**: The longest "shortest path" between two (reachable) nodes (Distance Matrix) – Structural Analysis

Hitachi Consulting
We make it happen. Better.

Distance Matrix
(Shortest Path Paris)

	A	B	C	D	E
A	-	8	∞	10	5
B	8	-	12	13	∞
C	11		-	20	∞
D	4	9	16	-	10
E	7	∞	∞	5	-

In this example (Directed Graph), the Graph Diameter is 20, which is the longest shortest path (that is the one from C to D)

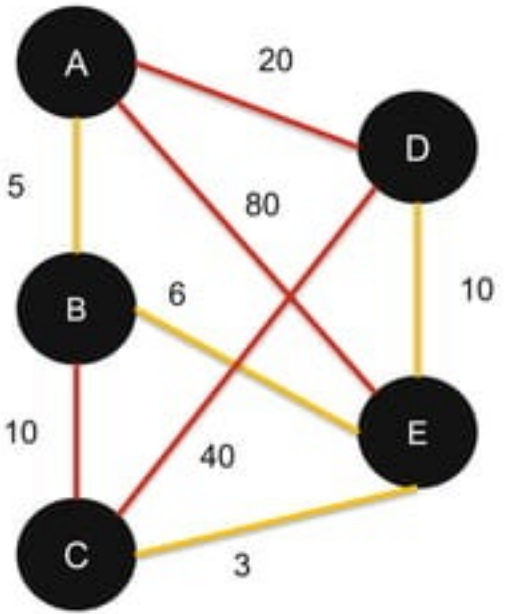
 | © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

Path Analytics & Graph Traversing

Concepts and operations

- **Path**: A set of (ordered) edges between node x and node y
- **Cycle**: A path where the start and the end nodes are the same
- **Trail**: A path with no repeated edges
- **Tour**: A cycle traversing all the nodes, only once.
- **Reachability**: Can we reach node D from node C?
- **Shortest path**: minimum steps (edges) between two nodes
  - Breadth-First Search
  - Dijkstra's algorithm
- **Best path** (weighted graph): path that minimize total weight
  - Optimize a given function
  - Satisfy given constrains
- **Graph Diameter**: The longest "shortest path" between two (reachable) nodes (Distance Matrix) – Structural Analysis
- **Minimum Spanning Trees**: edges that connect all the nodes with no cycles and minimum weight.



 | © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

Community Analytics

 | © Copyright 2015 Hitachi Consulting

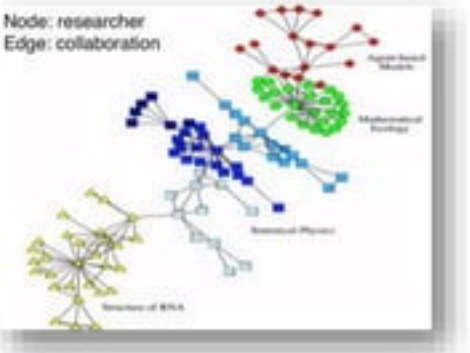
Hitachi Consulting
We make it happen. Better.

Community Analytics

Graph Clustering/Partitioning

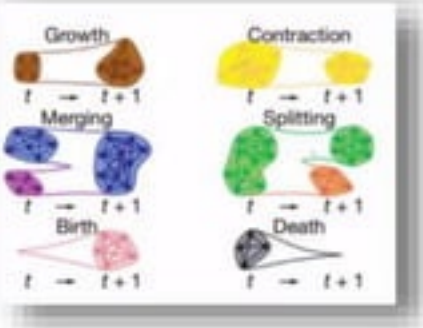
A dense subgraph (cluster) with in a graph, in which its nodes are more connected with a cluster than to the nodes outside the cluster

- Cohesion – Connectivity "within" the cluster is high
- Separation – Connectivity "between" clusters is low



Analytical Questions

- **Static** – Discover community
- **Static** – Describe interaction with a community
- **Static** – Describe interaction between communities
- **Temporal** – How a community emerged/dissolved?
- **Temporal** – Which communities are stable
- **Temporal** – Predict of a node will migrate to another community?



 | © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

Community Analytics

Graph Clustering/Partitioning

Finding Communities

Local Properties

- **n-Clique** (distance): largest subgraph that the maximum distance between each two nodes is <= n
- **n-Clans** (distance): an n-clique in which the largest distance between nodes in the subgraph is <=n
- **k-Core** (density): largest subgraph that each nodes is connected to at least k-nodes within the sub graph

Global Properties

Modularity –

- The fraction of the edges that fall within the given subgraph minus the expected such fraction if edges were distributed at random
- Reflects the **concentration of edges** within subgraph **compared with random distribution of edges** between all nodes regardless of subgraphs.

© Copyright 2015 Hitachi Consulting

Centrality Analytics

 | © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

Centrality Analytics

Vertex Importance Analysis

Connectivity Importance

- Simply, the **degree of node** X (in and **out** degrees).
  - I.e., the queen bees in a community (used for target marketing)

Closeness Importance

- Average **length** of all its **shortest paths**, compared to the averages of the other vertices (using Distance Matrix)
  - I.e., From vertex X, you can reach most of the other vertices quicker

Betweeness Importance

- The **fraction** of the **shortest paths** that X appears in.
  - I.e., if x is important, then most of the (shortest) paths between any two vertices in a graph pass through x (important underground station).

Vulnerability

- Vertex X belongs to the minimum node set that, if removed from the graph, the graph is **disconnected**.
  - Or, its removal will cause a high disruption in the network

Network Centralization (graph-level measure) – Measure of degree of variation of centrality score amongst the nodes of the network

Centrality Analytics

Vertex Importance Analysis

Page Rank - The importance (rank) of a vertex is computed as the total rank of all its adjacent edges (a.k.a Eigenvector Centrality).

- I.e., the importance of a given vertex is **not only how well-connected** it is, it is also **how well-connected its neighbours** are.
- Including a damping factor: the further the you go away the vertex, the less important it is on the rank of the vertex

PageRank can be interpreted as the probability to visit a page…

 | © Copyright 2015 Hitachi Consulting

Pattern Matching

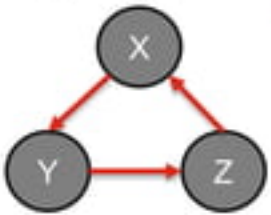
 | © Copyright 2015 Hitachi Consulting

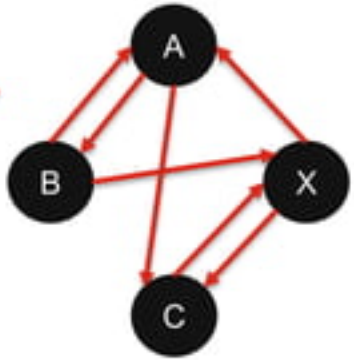
Hitachi Consulting
We make it happen. Better.

Pattern Matching

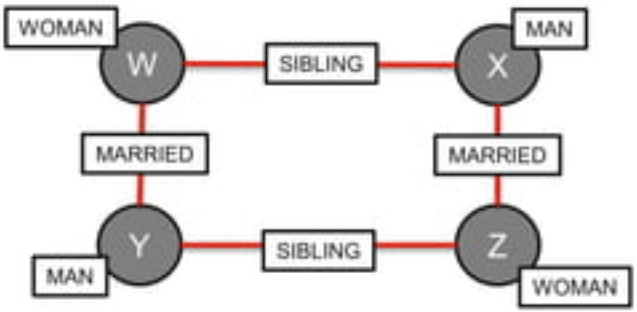
Graph Query

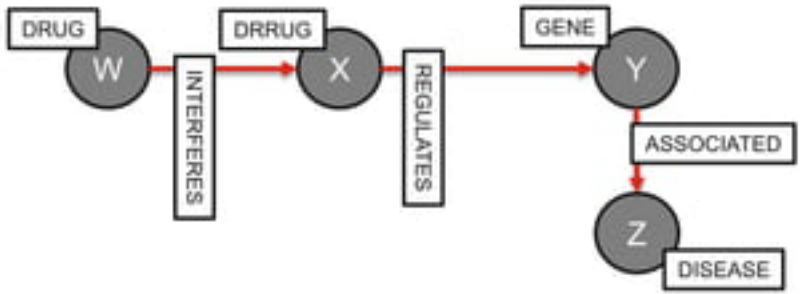
- Find the following patterns in a given graph





- Find the following patter in a given Property Model graph





 | © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

Pattern Matching

Applications

- **Banking** – Fraud Detection
- **Security** – Threat Detection
- **Bioinformatics & Biochemistry** – Association Analysis
- **Social Networks** – Job/Candidate suggestion
- **GPS & Smart Cities** – Traffic/Accident Analysis
- **Telecom** – Targeted Campaigning

 © Copyright 2015 Hitachi Consulting

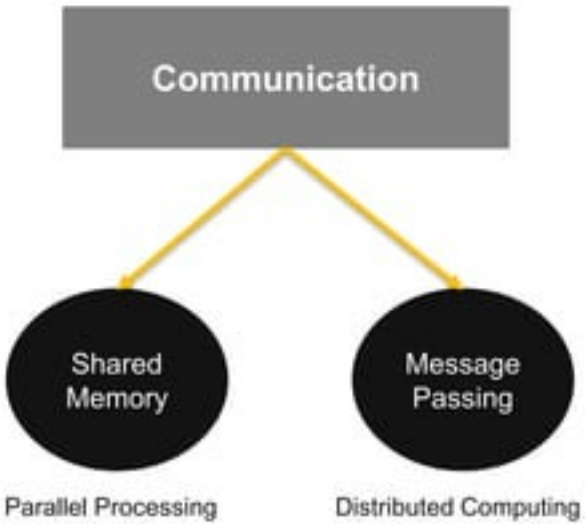
Parallel Programming Model for Graphs

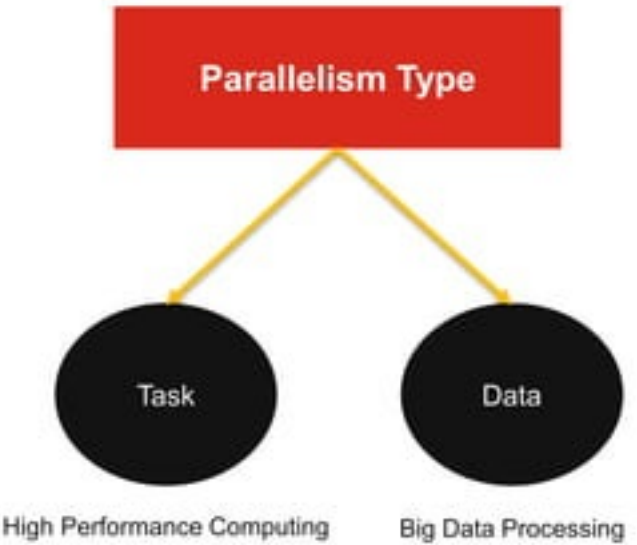
 | © Copyright 2015 Hitachi Consulting

Parallel Programming Model for Graphs

Graph Processing

Hitachi Consulting
We make it happen. Better.





 | © Copyright 2015 Hitachi Consulting

Parallel Programming Model for Graphs

Graph Processing

- **Data Parallelism –** Each compute node has a subset of graph vertex.
- **Message Passing –** A vertex can communicate (send/receive a message) to a vertex (in another compute node) if it has an outgoing edge to.
- **Processing of vertices is performed in parallel** – E.g., Bulk Synchronous Parallelism (BSP)

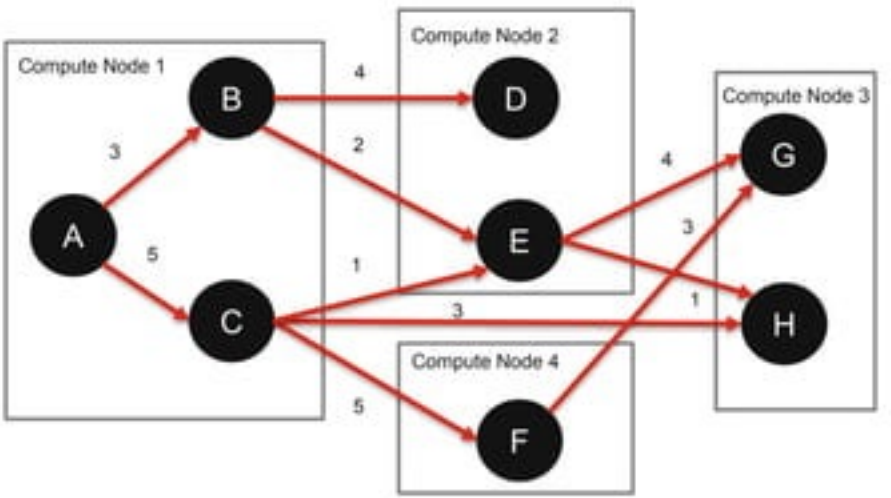
 | © Copyright 2015 Hitachi Consulting

Parallel Programming Model for Graphs

Graph Processing

- **Data Parallelism –** Each compute node has a subset of graph vertex.
- **Message Passing –** A vertex can communicate (send/receive a message) to a vertex (in another compute node) if it has an outgoing edge to.
- **Processing of vertices can is performed in parallel** – E.g., Bulk Synchronous Parallelism (BSP)

E.g.: Find the shortest path between A, H, in parallel



 | © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

Parallel Programming Model for Graphs

Graph Processing

Pregel - A System for Large Scale Graph Processing

- Published by Google
- Based on Bulk Synchronous Parallelism
  - Receive Messages from parent Nodes
  - Compute
  - Send Messages to Child Nodes
  - Pause & Synchronize

  } Superstep
- Example Application: PageRank

Graph Processing Tools:

- **Giraph –** HDFS, MapReduce, YARN (JAVA)
- **GraphX** – Spark, RDDs (Scala)

 | © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

Parallel Programming Model for Graphs

Graph Processing - GraphX

```
class Graph[VD, ED] {
  // Information about the Graph ===================================
  val numEdges: Long
  val numVertices: Long
  val inDegrees: VertexRDD[Int]
  val outDegrees: VertexRDD[Int]
  val degrees: VertexRDD[Int]
  // views of the graph as collections =============================
  val vertices: VertexRDD[VD]
  val edges: EdgeRDD[ED]
  val triplets: RDD[EdgeTriplet[VD, ED]]
  // functions for caching graphs ==================================
  def persist(newLevel: StorageLevel = StorageLevel.MEMORY_ONLY): Graph[VD, ED]
  def cache(): Graph[VD, ED]
  def unpersistVertices(blocking: Boolean = true): Graph[VD, ED]
  // change the partitioning heuristic ============================
  def partitionBy(partitionStrategy: PartitionStrategy): Graph[VD, ED]
  // transform vertex and edge attributes ==========================
  def mapVertices[VD2](map: (VertexId, VD) => VD2): Graph[VD2, ED]
  def mapEdges[ED2](map: Edge[ED] => ED2): Graph[VD, ED2]
  def mapEdges[ED2](map: (PartitionID, Iterator[Edge[ED]]) => Iterator[ED2]): Graph[VD, ED2]
  def mapTriplets[ED2](map: EdgeTriplet[VD, ED] => ED2): Graph[VD, ED2]
  def mapTriplets[ED2](map: (PartitionID, Iterator[EdgeTriplet[VD, ED]]) => Iterator[ED2])
    : Graph[VD, ED2]
  def reverse: Graph[VD, ED]
  def subgraph(epred: EdgeTriplet[VD,ED] => Boolean = (x => true),
      vpred: (VertexId, VD) => Boolean = ((v, d) => true)): Graph[VD, ED]
```

```
  // Modify the graph structure ====================================
  def mask[VD2, ED2](other: Graph[VD2, ED2]): Graph[VD, ED]
  def groupEdges(merge: (ED, ED) => ED): Graph[VD, ED]
  // Join RDDs with the graph ======================================
  def joinVertices[U](table: RDD[(VertexId, U)])(mapFunc: (VertexId, VD, U) => VD)
        : Graph[VD, ED]
  def outerJoinVertices[U, VD2](other: RDD[(VertexId, U)])
      (mapFunc: (VertexId, VD, Option[U]) => VD2) : Graph[VD2, ED]

// Aggregate information about adjacent triplets ===================
  def collectNeighborIds(edgeDirection: EdgeDirection): VertexRDD[Array[VertexId]]
  def collectNeighbors(edgeDirection: EdgeDirection): VertexRDD[Array[(VertexId, VD)]]
  def aggregateMessages[Msg: ClassTag](
      sendMsg: EdgeContext[VD, ED, Msg] => Unit,
      mergeMsg: (Msg, Msg) => Msg,
      tripletFields: TripletFields = TripletFields.All)
    : VertexRDD[A]
  // Iterative graph-parallel computation ==========================
  def pregel[A](initialMsg: A, maxIterations: Int, activeDirection: EdgeDirection){
      vprog: (VertexId, VD, A) => VD,
      sendMsg: EdgeTriplet[VD, ED] => Iterator[(VertexId,A)],
      mergeMsg: (A, A) => A)
    : Graph[VD, ED]
  // Basic graph algorithms ========================================
  def pageRank(tol: Double, resetProb: Double = 0.15): Graph[Double, Double]
  def connectedComponents(): Graph[VertexId, ED]
  def triangleCount(): Graph[Int, ED]
  def stronglyConnectedComponents(numIter: Int): Graph[VertexId, ED]
}
```


© Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

Applied Graph Analytics

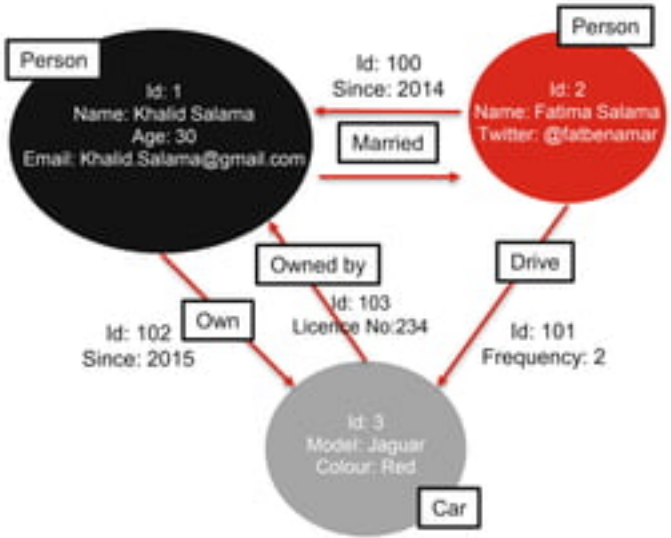
 | © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

Graph Databases

NoSQL Graph Stores

- Represent data in **graphical structures**: Nodes and Edges.
- **Nodes** represent **entities**, **Edges** represent **relationships** between entities.
- Relationships are **directed**, semantics of the direction is up to the application. E.g. "Married" is reflexive, "Owns" is not.
- Each Node/Edge has a set of **Key/Value properties**
- Each Node/Edge has a **label** (type of entity/relationship)
- **Optimized** to process **graph-related queries** and analytics.



- Example Tools
  - Neo4j
  - OrientDB
  - Titan
  - Apache Giraph
  - Microsoft Graph Engine (Trinity)

Real-world Scenarios

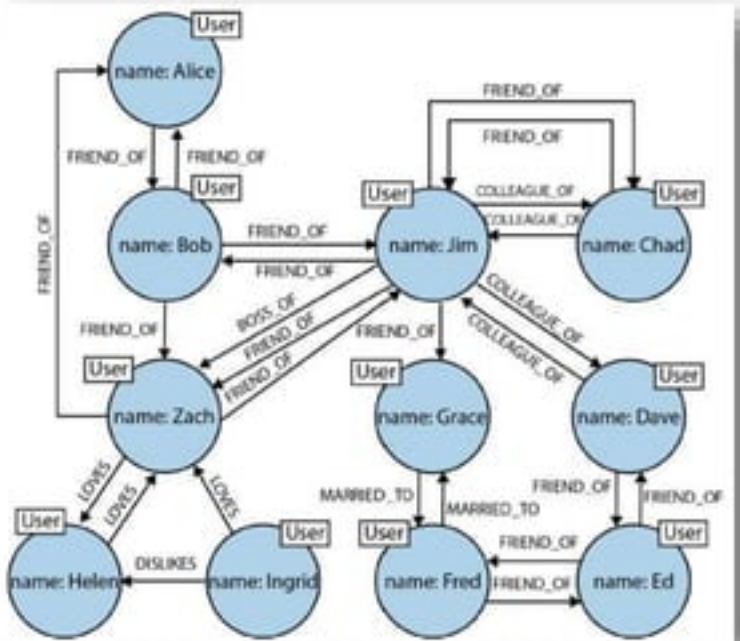
- Social Networks
- Network and IT Operations
- Fraud Detection
- Digital Assets Management

 | © Copyright 2015 Hitachi Consulting

Graph Databases

NoSQL Graph Stores

Hitachi Consulting
We make it happen. Better.



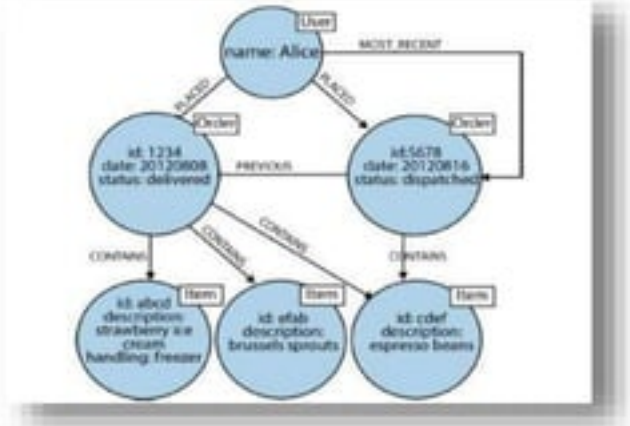


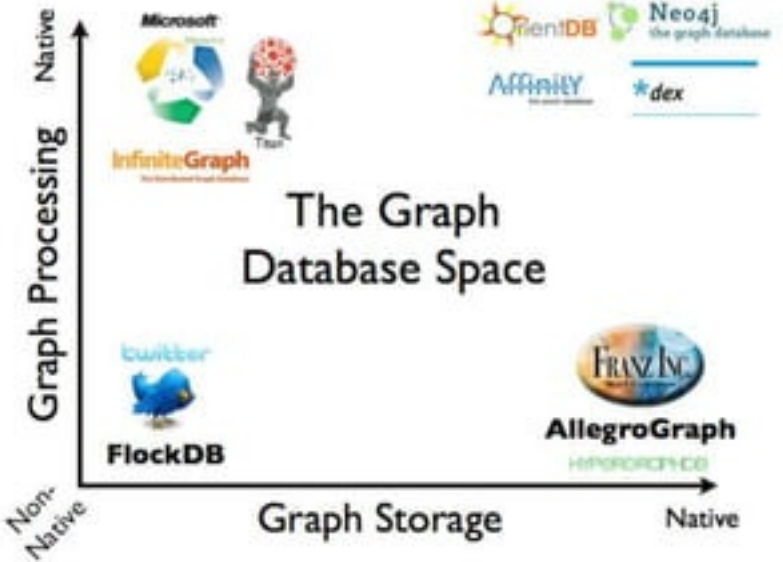
Table 2-1. Finding extended friends in a relational database versus efficient finding in Neo4j

Depth	RDBMS execution time(s)	Neo4j execution time(s)	Records returned
2	0.016	0.01	~2500
3	30.267	0.168	~110,000
4	1543.505	1.359	~600,000
5	Unfinished	2.132	~800,000

Graph Databases

NoSQL Graph Stores

index-free adjacency; connected nodes physically "point" to each other in the database



Any database *behaves* like a graphDB; exposes a graph data model through CRUD operations

Graphs are serialized in any database; Relational, Document, or objectDBs

Storage is designed and optimize to store, process, and query graph data structures

 | © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

Applied Graph Analytics

Neo4j Graph Database

- Most Popular GraphDB (according to db-engines).
- Free Community Edition and Commercial Enterprise Edition.
- Native Graph Processing and Storage.
- Uses Cypher Query Language (CQL).
- Scalability (Redundancy and Load Balancing) with High Availability (HA) package.
- Read capacity of HA cluster increases linearly with the number of servers.
- Can commit 10K of writes per second while maintaining fully ACID transactions.

 | © Copyright 2015 Hitachi Consulting

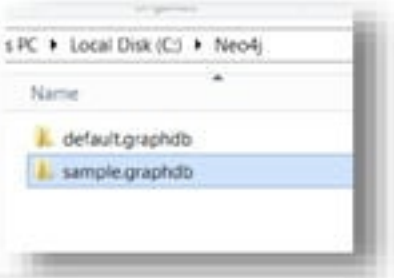
Applied Graph Analytics

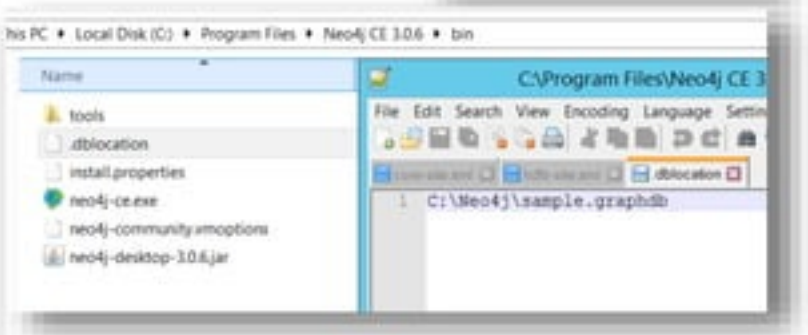
Neo4j Graph Database

Hitachi Consulting
We make it happen. Better.

Create a Database

1. Create a folder in your file system (e.g. sample.graphdb)
2. Set the location of the database in .dblocation
3. Lunch neo4j





 | © Copyright 2015 Hitachi Consulting

Applied Graph Analytics

Neo4j Graph Database

Create a node

```
create (<Id>:<label>{<Property>:"Value",…})
```

Example

```
create (p1:Person{name:"khalid", age:"31", gender:"male"})
```

 | © Copyright 2015 Hitachi Consulting

Applied Graph Analytics

Neo4j Graph Database

Create an edge

create ((<nodeId>)
-[<edgeId>:<label>{<Property>:"Value",…}]->
(<nodeId>))

Example

create ((p1)-[e1:follows{datetime:"2010-10-05"}]->(p2))

 | © Copyright 2015 Hitachi Consulting

Applied Graph Analytics

Neo4j Graph Database

Retrieve nodes/edges

match (<pattern>) return (<objects>)

Example

match (p:perons) return p
Match (p1)-[r]->(p1) return p1,p2, r

 | © Copyright 2015 Hitachi Consulting

Applied Graph Analytics

Neo4j Graph Database

Update graph

match (<pattern>) merge (<objects>)

match (<pattern>) set (<object>.property = value)

Example

match (p:perons{name="khalid Salama") merge (p)-[:marriedTo]->(m:perons{name="Fatima Zahra"})

match (p:person) where name = "khalid Salame" set job="IT Manager"

 | © Copyright 2015 Hitachi Consulting

Applied Graph Analytics

Neo4j Graph Database

Delete nodes/edges

match (<pattern>) delete (<objects>)

Example

match (n)-[e]-() delete n,e

 | © Copyright 2015 Hitachi Consulting

Applied Graph Analytics

Neo4j Graph Database

Import csv Data to Neo4j

Source	Target	Distance
A	B	4
A	C	5
B	D	5
C	B	6

```
LOAD CSV WITH HEADER <filepath>.csv AS line
MERGE (x:city{name:line.Source})
MAEGE (y:city{name:line.Target})
MERGE (x)-[:To{distance=line.Distance}]->(y)
```

 | © Copyright 2015 Hitachi Consulting

Applied Graph Analytics

Neo4j Graph Database

Basic Queries

```
//Counting the number of nodes
match (n:Label)
return count(n)
//Counting the number of edges
match (n:Label)-[r]->()
return count(r)
//Finding leaf nodes:
match (n:Label)-[r:TO]->(m)
where not ((m)-->())
return m
//Finding root nodes:
match (m)-[r:TO]->(n:Label)
where not (()-->(m))
return m
//Finding triangles:
match (a)-[:TO]->(b)-[:TO]->(c)-[:TO]->(a)
return distinct a, b, c
```

```
//Finding 2nd neighbors of D:
match (a)-[:TO*..2]-(b)
where a.Name='D'
return distinct a, b
//Finding the types of a node:
match (n)
where n.Name = 'Egypt'
return labels(n)
//Finding the label of an edge:
match (n {Name: 'Egypt'})<-[r]-()
return distinct type(r)
//Finding all properties of a node:
match (n:Actor)
return * limit 20
//Finding loops:
match (n)-[r]->(n)
return n, r limit 10
//Finding multigraphs:
match (n)-[r1]->(m), (n)-[r2]-(m)
where r1 <> r2
return n, r1, r2, m limit 10
//Finding the induced subgraph given a set of nodes:
match (n)-[r:TO]-(m)
where n.Name in ['A', 'B', 'C', 'D', 'E'] and m.Name in ['A', 'B', 'C', 'D', 'E']
return n, r, m
```

Applied Graph Analytics

Neo4j Graph Database

Hitachi Consulting
We make it happen. Better.

Path Analysis

```
//Finding paths between specific nodes:
match p=(a)-[:TO]-(c)
where a.Name='H' and c.Name='P'
return p limit 1
//Finding the length between specific nodes:
match p=(a)-[:TO*]-(c)
where a.Name='H' and c.Name='P'
return length(p) limit 1
//Finding a shortest path between specific nodes:
match p=shortestPath((a)-[:TO*]-(c))
where a.Name='A' and c.Name='P'
return p, length(p) limit 1
//All Shortest Paths with Path Conditions:
match p = allShortestPaths((source)-[r:TO*]->(destination))
where source.Name='A' and destination.Name = 'P' and
length(nodes(p)) > 5
return extract(n in NODES(p)| n.Name) as Path, length(p) as
PathLenght
```

```
//Diameter of the graph:
match (n:Label), (m:Label)
where n <> m
with n, m
match p=shortestPath((n)-[*]->(m))
return n.Name, m.Name, length(p)
order by length(p) desc limit 1
//Extracting and computing with node and properties:
match p=(a)-[:TO*]-(c)
where a.Name='H' and c.Name='P'
return extract(n in nodes(p)|n.Name) as Path, length(p) as pathLength,
reduce(s=0, e in relationships(p)| s + toInt(e.dist)) as pathDist limit 1
//Graph not containing a selected node:
match (n)-[r:TO]->(m)
where n.Name <> 'D' and m.Name <> 'D'
return n, r, m
match (d {Name:'D'})-[:TO]-(b)<-[:TO]-(root)
where not((root)<--())
return (root)
//Graph not containing a selected neighborhood:
match (a {Name: 'F'})-[:TO*..2]-(b)
with collect(distinct b.Name) as MyList
match (n)-[r:TO]->(m)
where not(n.Name in MyList) and not (m.Name in MyList)
return distinct n, r, m
```

 | © Copyright 2015 Hitachi Consulting

Applied Graph Analytics

Neo4j Graph Database

Connectivity Analysis

```
// Find the outdegree of all nodes
match (n:Label)-[r]->()
return n.Name as Node, count(r) as Outdegree
order by Outdegree
union
match (a:Label)-[r]->(leaf)
where not((leaf)-->())
return leaf.Name as Node, 0 as Outdegree
// Find the indegree of all nodes
match (n:Label)<-[r]-()
return n.Name as Node, count(r) as Indegree
order by Indegree
union
match (a:Label)<-[r]-(root)
where not((root)<--())
return root.Name as Node, 0 as Indegree
```

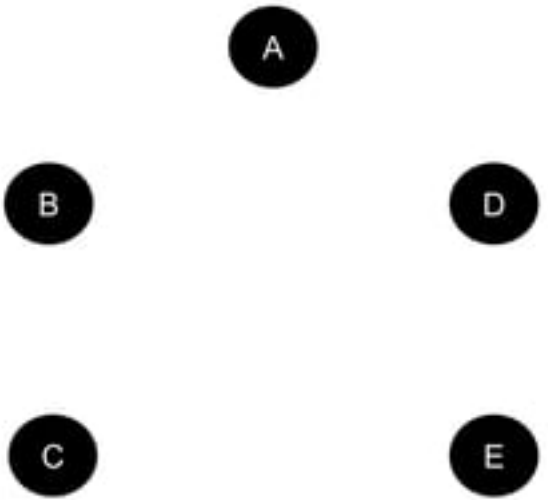
```
//Find the degree of all nodes
match (n:Label)-[r]-()
return n.Name, count(distinct r) as degree
order by degree
// Find degree histogram of the graph
match (n:Label)-[r]-()
with n as nodes, count(distinct r) as degree
return degree, count(nodes) order by degree asc
//Save the degree of the node as a new node property
match (n:Label)-[r]-()
with n, count(distinct r) as degree
set n.deg = degree
return n.Name, n.deg
// Construct the Adjacency Matrix of the graph
match (n:Label), (m:Label)
return n.Name, m.Name,
case
when (n)-->(m) then 1
else 0
end as value
```

 | © Copyright 2015 Hitachi Consulting

Applied Graph Analytics

Neo4j Graph Database - Example

Hitachi Consulting
We make it happen. Better.

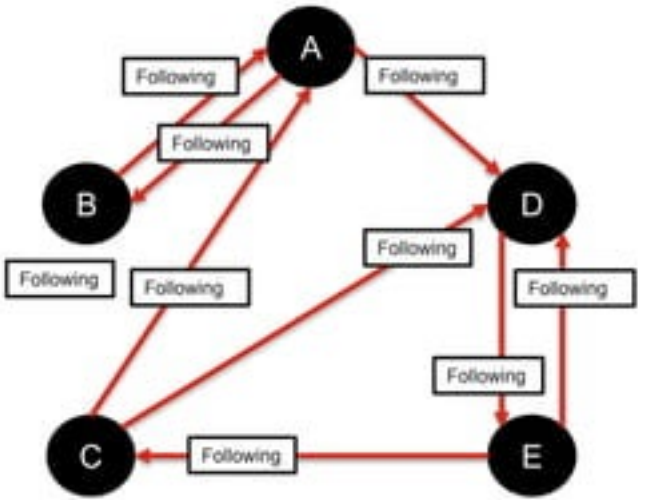


 | © Copyright 2015 Hitachi Consulting

Applied Graph Analytics

Neo4j Graph Database - Example

Hitachi Consulting
We make it happen. Better.

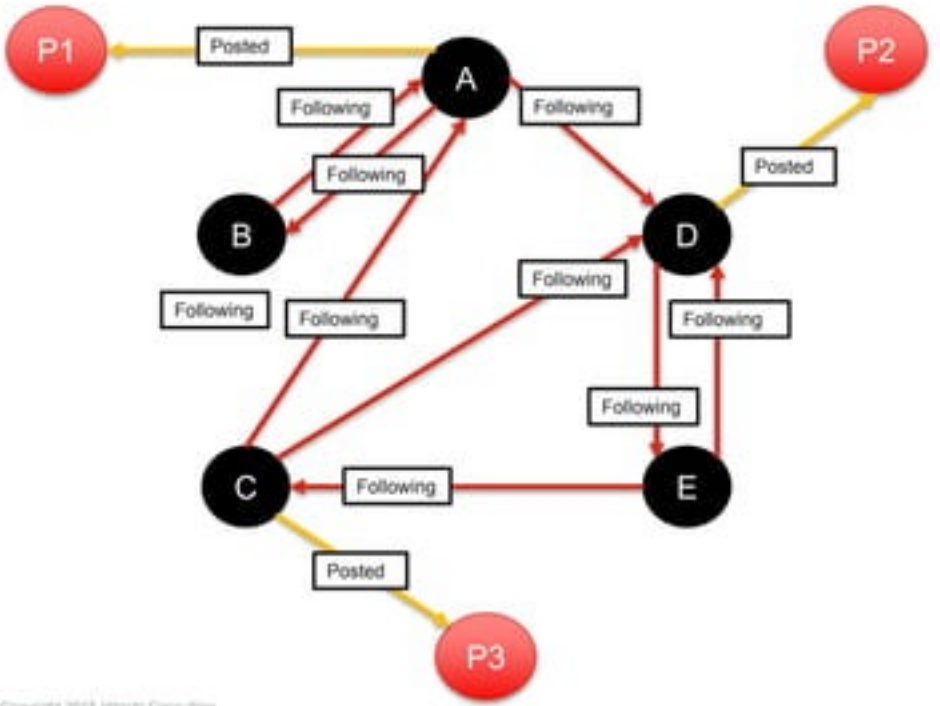


 | © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

Applied Graph Analytics

Neo4j Graph Database - Example

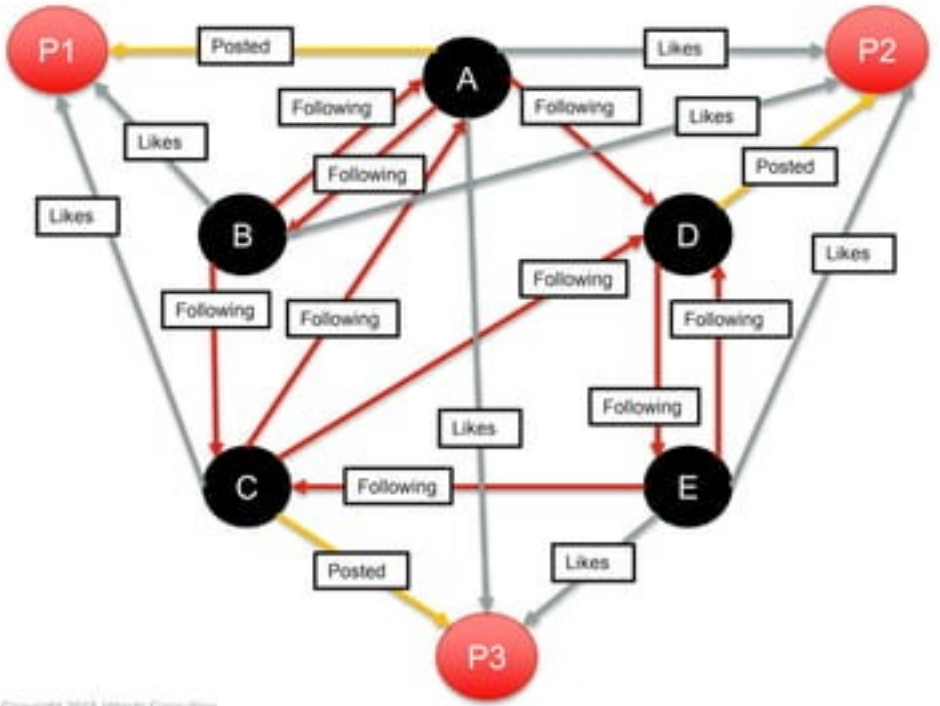


 | © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

Applied Graph Analytics

Neo4j Graph Database - Example



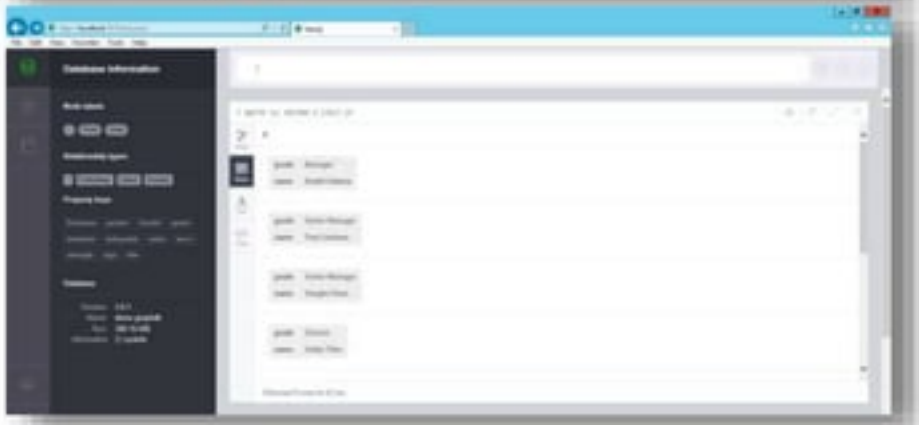
 | © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

Applied Graph Analytics

Neo4j Graph Database - Example

```
CREATE

(a:User{name:"Khalid Salama", grade:"Manager"}),
(b:User{name:"Paul Lineham", grade:"Senior Manager"}),
(c:User{name:"Vaughn Rees", grade:"Senior Manager"}),
(d:User{name:"Sutha Thiru", grade:"Director"}),
(e:User{name:"Mark Hill", grade:"VP"}),

(a)-[:Following{since:'2014'}]->(d),
(a)-[:Following{since:'2014'}]->(b),
(b)-[:Following{since:'2010'}]->(a),
(d)-[:Following{since:'2011', strength:"high"}]->(e),
(e)-[:Following{since:'2014'}]->(d),
(e)-[:Following{since:'2015'}]->(c),
(c)-[:Following]->(d),
(c)-[:Following{since:'2013', strength:"low"}]->(a),
(b)-[:Following]->(c),

(p1:Post{title:"post 1", lastupdate:"01/01/2016", tags:['sports','life style']}),
(p2:Post{title:"post 2", lastupdate:"03/05/2015"}),
(p3:Post{title:"post 3", lastupdate:"121/7/2015", tags:['economics','politcs']}),

(a)-[:Posted]->(p1),
(d)-[:Posted]->(p2),
(c)-[:Posted]->(p3),

(b)-[:Liked]->(p1),
(c)-[:Liked]->(p1),
(a)-[:Liked]->(p2),
(b)-[:Liked]->(p2),
(e)-[:Liked]->(p2),
(a)-[:Liked]->(p3),
(e)-[:Liked]->(p3)
```



 | © Copyright 2015 Hitachi Consulting

Applied Graph Analytics

Neo4j Graph Database - Example

```
//fetch one node
MATCH (u:User{name:"Khalid Salama"}) RETURN u

// fetch an attribute of a node
MATCH (u:User{name:"Khalid Salama"}) RETURN u.grade

// fetch nodes by conditions
MATCH (u:User{grade:"Senior Manager"}) RETURN u
--
MATCH (u:User)
WHERE u.grade = 'Senior Manager'
RETURN u
--
MATCH (u:User)
WHERE u.name =~ "Sutha.*" // START WITH, END WITH, CONTAIN, IN [,].
RETURN u
--
MATCH ()-[r:Posted]->(p:Post)
WHERE 'sports' IN p.tags
RETURN p

// Whom khalid is following?
MATCH (x:User{name:"Khalid Salama"})-[r:Following]->(y:User)
RETURN x,r,y

// Who is Following Khalid
MATCH (x:User{name:"Khalid Salama"})<-[r:Following]-(y:User)
RETURN x,r,y

// Update
MERGE (u:User { name:"Khalid Salama" })
SET u.practice = "Data Insights & Analytics"
RETURN u
```



```
// Get Count of Posts
MATCH (p:Post) RETURN COUNT(p)
-- Get User Count By Grade
MATCH (u:User) RETURN u.grade, COUNT(u)
-- Get User and Followers
MATCH (u:User)<-[:Following]-(f:User)
RETURN u.name AS User,COLLECT(f.name) AS follows,COUNT(f) AS Total

// Constraint
CREATE CONSTRAINT ON (u:User) ASSERT u.name IS UNIQUE
-- Index
CREATE INDEX ON :User(grade)

// Get users following each other
MATCH (u1:User)-[:Following]->(u2:User)-[:Following]->(u1)
RETURN u1.name,u2.name

// Get Users likes a post posted by a follower
MATCH (u:User)-[:Liked]->(p:Post)<-[:Posted]-(u2:User)-[:Following]->(u)
RETURN u,p,u2

// Get Following of Following
MATCH (u:User)-[:Following]->()-[:Following]->(u2:User)
Return u.name,COLLECT(DISTINCT u2.name)

// Get User with max 3 steps from Paul
MATCH (u:User)-[:Following*..3]->(us:User{name:"Paul Lineham"})
Return u

// Shortest path

MATCH
        (u1:User{name:"Mark Hill"}),
        (u2:User{name:"Paul Lineham"}),
        p=SHORTESTPATH((u1)-[:Following*..10]->(u2))
RETURN p

-- Get nodes having a property
MATCH(p)
WHERE EXSITS(p.tags)
```

Hitachi Consulting
We make it happen. Better.

http://neo4j.com/docs/developer-manual/current/#cypher-query-lang

Useful Resources

Hitachi Consulting
We make it happen. Better.

- **Coursera** – Graph Analytics for Big Data
https://www.coursera.org/learn/big-data-graph-analytics/home/welcome

- **Coursera** – Data Manipulation at Scale (Lessons 21-24)
https://www.coursera.org/learn/data-manipulation/home/week/4

- **Neo4j** – Getting Started Tutorials
https://neo4j.com/developer/get-started

- **Apache Spark** – GraphX Documentation
http://spark.apache.org/docs/latest/graphx-programming-guide.html

 | © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

My Background

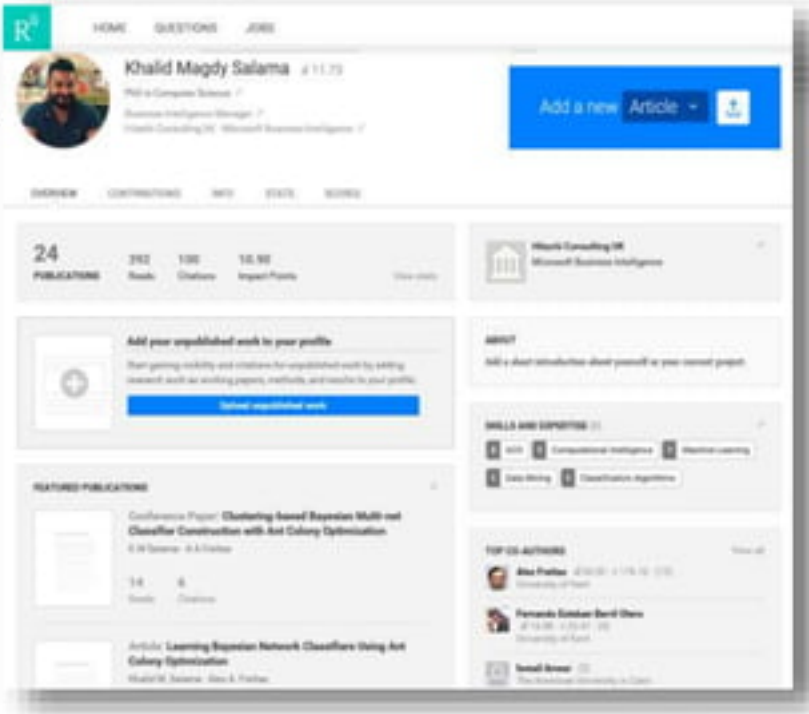
Applying Computational Intelligence in Data Mining

- Honorary Research Fellow, School of Computing , University of Kent.
- Ph.D. Computer Science, University of Kent, Canterbury, UK.
- M.Sc. Computer Science , The American University in Cairo, Egypt.

- 25+ published journal and conference papers, focusing on:
  - *classification rules induction,*
  - *decision trees construction,*
  - *Bayesian classification modelling,*
  - *data reduction,*
  - *instance-based learning,*
  - *evolving neural networks*, and
  - *data clustering*

- **Journals**: *Swarm Intelligence, Swarm & Evolutionary Computation, , Applied Soft Computing, and Memetic Computing.*

- **Conferences**: *ANTS, IEEE CEC, IEEE SIS, EvoBio, ECTA, IEEE WCCI and INNS-BigData.*



ResearchGate.org

 | © Copyright 2015 Hitachi Consulting

Hitachi Consulting
We make it happen. Better.

Thank you!

 | © Copyright 2015 Hitachi Consulting